

WEEK 1-Practicals

Algorithm & Flowchart

1. Convert Fahrenheit to Centigrade

Algorithm: Fahrenheit to Centigrade Conversion

1. Start
2. Read temperature in Fahrenheit (F)
3. Calculate Centigrade using formula:

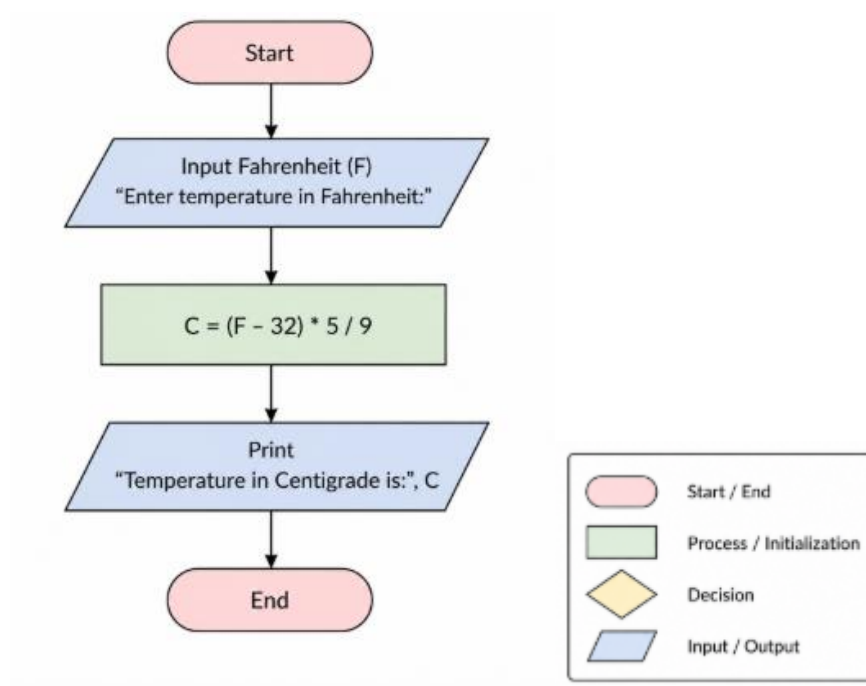
$$C = (F - 32) \times \frac{5}{9}$$

4. Display temperature in Centigrade
5. Stop

Python Code: Fahrenheit to Centigrade

```
# Program to convert Fahrenheit to Centigrade
# Input temperature in Fahrenheit
F = float(input("Enter temperature in Fahrenheit: "))
# Conversion formula
C = (F - 32) * 5 / 9
# Display result
print("Temperature in Centigrade is:", C)
```

Flow chart : Fahrenheit to Centigrade



2. Multiplication Table of 5 (Using Loop)

Algorithm:

1. **Start**
2. Initialize $n = 5$
3. Initialize $i = 1$
4. **While** $i \leq 10$
 - Calculate $\text{result} = n \times i$
 - Display $n \times i = \text{result}$
 - Increment $i = i + 1$
5. **Stop**

Python Code:

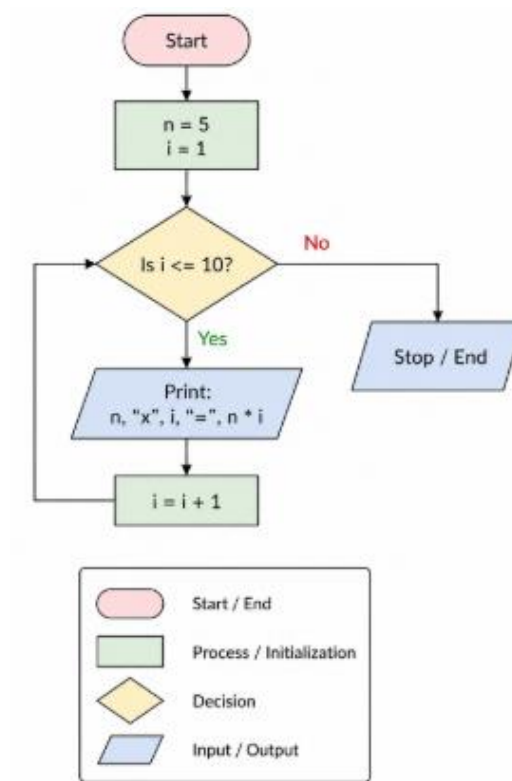
Program to display multiplication table of 5 using loop

$n = 5$

for i in range(1, 11):

 print(n , "x", i , "=", $n * i$)

Flow chart:



3. Finding the Maximum of Three Forces

Algorithm:

Start

Read three forces: F_1, F_2, F_3

If $F_1 \geq F_2$ **and** $F_1 \geq F_3$

→ $\text{Max} = F_1$

Else if $F2 \geq F1$ and $F2 \geq F3$

→ Max = F2

Else

→ Max = F3

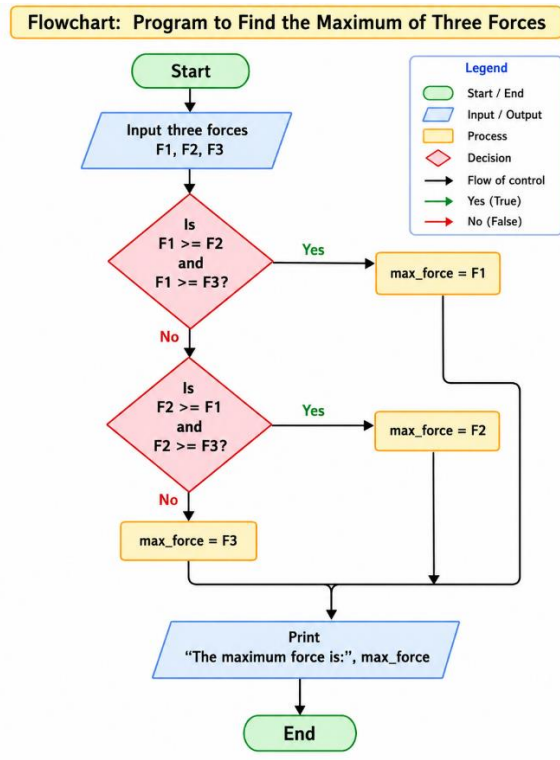
Display the maximum force

Stop

Python code :

```
# Program to find the maximum of three forces
# Input three forces
F1 = float(input("Enter Force 1: "))
F2 = float(input("Enter Force 2: "))
F3 = float(input("Enter Force 3: "))
# Decision making to find maximum force
if F1 >= F2 and F1 >= F3:
    max_force = F1
elif F2 >= F1 and F2 >= F3:
    max_force = F2
else:
    max_force = F3
# Display the result
print("The maximum force is:", max_force)
```

Flow chart:



PYTHON PROGRAMMING LAB MANUAL

Week 2 — Introduction to Python and Basic Constructs

Program	Mechanical Engineering	Semester	IV
Course Name	Programming Concepts in Mechanical Engineering	Course Code	25ME34I
Week	2	Duration	3 Hours (Practice Session)
Course Outcome	CO-1: Use appropriate syntaxes while writing program.	Program Outcomes	PO-1, PO-2

Learning Objectives

- Get comfortable with Python syntax, variables, and basic data types (int, float, string, boolean).
- Use input() and print() to build simple interactive programs.
- Apply type conversion functions (int(), float(), str()) correctly.
- Use arithmetic and assignment operators to solve small engineering calculations.

Quick Theory Recap

- Python environment: code can be written and run in Anaconda, Jupyter Notebook, or an IDE such as PyCharm.
- Data types: int (whole numbers), float (decimals), string (text), boolean (True / False).
- input() always returns a string — convert it with int() or float() before doing arithmetic.
- print() displays output; multiple values can be separated by commas.
- Arithmetic operators: + - * / // % **. Assignment operators: = += -= *= /=.

Practical Exercises

Exercise 1: Basic Arithmetic Operations

Aim: Write a Python program to accept two numbers from the user and perform addition, subtraction, multiplication, and division.

Program:

```
# Program to perform arithmetic operations

# Accept two numbers from the user
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

# Arithmetic operations
addition = a + b
subtraction = a - b
```

```

multiplication = a * b

# Check to avoid division by zero
if b != 0:
    division = a / b
else:
    division = "Not possible (division by zero)"

# Display results
print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)

```

Sample Output

```

Enter first number: 10
Enter second number: 4
Addition: 14.0
Subtraction: 6.0
Multiplication: 40.0
Division: 2.5

```

Explanation: input() collects text from the keyboard; float() converts it to a decimal number so arithmetic can be performed. An if-else check prevents a crash if the user enters 0 as the second number.

Exercise 2: Output Speed and Torque in a Gear System

Aim: Calculate output speed and output torque of a gear system from user-entered input speed, input torque, and gear ratio.

Concept:

- Gear Ratio (GR) = Input Speed ÷ Output Speed
- Output Speed = Input Speed ÷ Gear Ratio
- Output Torque = Input Torque × Gear Ratio

Program:

```

# Program to calculate output speed and torque in a gear system

# Input values from user
input_speed = float(input("Enter input speed (RPM): "))
input_torque = float(input("Enter input torque (Nm): "))
gear_ratio = float(input("Enter gear ratio: "))

# Calculations using basic operators
output_speed = input_speed / gear_ratio
output_torque = input_torque * gear_ratio

# Display results
print("Output Speed (RPM):", output_speed)

```

```
print("Output Torque (Nm):", output_torque)
```

Sample Output

```
Enter input speed (RPM): 1200
Enter input torque (Nm): 50
Enter gear ratio: 4
Output Speed (RPM): 300.0
Output Torque (Nm): 200.0
```

*Explanation: This is a real mechanical-engineering use of arithmetic operators — the same / and * used above, applied to a gearbox calculation.*

Exercise 3: Working with Variables

Aim: Store values of different data types in variables and use them in expressions.

3(a) Name, Age, Height

```
name = "Rahul"
age = 18
height = 5.6
print(name, age, height)
```

Sample Output

```
Rahul 18 5.6
```

Explanation: name is a string, age is an int, height is a float — Python variables do not need a declared type.

3(b) Sum and Product

```
a = 5
b = 10
print(a + b)
print(a * b)
```

Sample Output

```
15
50
```

*Explanation: + adds the two values; * multiplies them.*

3(c) Area of a Circle

```
pi = 3.14
r = 7
```

```
area = pi * r * r
print(area)
```

Sample Output

```
153.86
```

Explanation: Implements the formula $Area = \pi r^2$.

Exercise 4: Boolean Data Type

Aim: Demonstrate the boolean data type and its use in engineering decision-making.

Program:

```
# Boolean example: checking if a machine is running
machine_running = True
temperature_ok = True

# A boolean expression combining two conditions
safe_to_operate = machine_running and temperature_ok

print("Machine running:", machine_running)
print("Temperature OK:", temperature_ok)
print("Safe to operate:", safe_to_operate)
```

Sample Output

```
Machine running: True
Temperature OK: True
Safe to operate: True
```

Explanation: A boolean variable holds only True or False. In engineering programs, booleans are used to represent machine/sensor status and to combine multiple safety conditions before an action is taken.

Exercise 5: Type Conversion

Aim: Convert between data types using `int()`, `float()`, and `str()`, and observe the results.

5(a) Integer + Float

```
a = 5
b = 2.5
c = a + b
print(c, type(c))
```

Sample Output

```
7.5 <class 'float'>
```

Explanation: Python automatically (implicitly) converts the int to a float so the addition can be performed.

5(b) String to Integer

```
a = "25"
b = int(a)
print(b + 5)
```

Sample Output

```
30
```

Explanation: A string coming from input() must be explicitly converted with int() before it can be used in arithmetic.

5(c) Float to Integer

```
x = int(98.6)
print(x)
```

Sample Output

```
98
```

Explanation: int() truncates (removes) the decimal part — it does not round the number.

Exercise 6: Bonus: Input-Based Decision Problems (Preview of Week 3)

Note: These two problems use the if-else statement, which is formally covered in Week 3 (Control Structures). They are included here only as an optional extension for students who finish early — mastering input() and type conversion first is the priority for this week.

6(a) Pass / Fail

```
marks = int(input())

if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

Sample Output

```
40
Pass
```

6(b) Voting Eligibility

```
age = int(input())
```



```
if age >= 18:
    print("Eligible")
else:
    print("Not Eligible")
```

Sample Output

```
20
Eligible
```

Self-Practice (To Be Completed Individually)

Attempt the following on your own and get your output verified by the faculty:

- Write a Python program that accepts the length and breadth of a rectangular plate and prints its area and perimeter.
- Write a Python program that converts a temperature entered in Celsius to Fahrenheit ($F = C \times 9/5 + 32$).
- Write a Python program that stores a boolean value indicating whether a pressure vessel has passed a safety test, and prints an appropriate message.
- Write a Python program that accepts a value as a string, converts it to a float, and prints its data type before and after conversion.

Reference: Week 2 syllabus content (25ME34I — Programming Concepts in Mechanical Engineering, DTE Karnataka).

PYTHON PROGRAMMING LAB MANUAL

Week 3 — Control Structures (Conditional Statements and Loops)

Program	Mechanical Engineering	Semester	IV
Course Name	Programming Concepts in Mechanical Engineering	Course Code	25ME34I
Week	3	Duration	3 Hours (Practice Session)
Course Outcome	CO-1: Use appropriate syntaxes while writing program.	Program Outcomes	PO-1, PO-2, PO-4

Learning Objectives

- Write decision-making programs using if, if-else, elif, and nested if statements.
- Use for loops and while loops to repeat a set of instructions.
- Apply loop control statements — break, continue, and pass — to control loop execution.
- Combine conditionals and loops to solve small engineering problems.

Quick Theory Recap

- if / elif / else: run a block of code only when a condition is True; elif checks additional conditions in order.
- Nested if: an if statement placed inside another if (or else) block, used when a decision depends on more than one condition, checked in stages.
- for loop: repeats a block a fixed number of times, typically over range() or a sequence.
- while loop: repeats a block as long as a condition remains True — used when the number of repetitions is not known in advance.
- break: exits the loop immediately. continue: skips the rest of the current iteration and moves to the next. pass: a placeholder that does nothing (used to keep syntactically valid empty blocks).

Practical Exercises

Exercise 1: Even or Odd Number

Aim: Write a Python program to check whether a number entered by the user is even or odd.

Program:

```
# Program to check whether a number is even or odd

# Input from user
num = int(input("Enter a number: "))

# Decision making
if num % 2 == 0:
    print("The number is Even")
```

```
else:
    print("The number is Odd")
```

Sample Output

```
Enter a number: 17
The number is Odd
```

Explanation: % is the modulus operator — it gives the remainder of division. If the remainder is 0, the number is Even; otherwise it is Odd.

Exercise 2: Multiplication Table Using a for Loop

Aim: Print the multiplication table of a number entered by the user using a for loop.

Program:

```
# Program to print multiplication table of a given number

# Input from user
num = int(input("Enter a number: "))

# Using for loop
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Sample Output

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

*Explanation: range(1, 11) runs the loop from 1 to 10; num * i calculates the multiplication; the for loop repeats the print statement 10 times.*

Exercise 3: Factorial of a Number Using a while Loop

Aim: Calculate the factorial of a number entered by the user using a while loop.

Concept:

- Factorial of n ($n!$) = $1 \times 2 \times 3 \times \dots \times n$

Program:

```
# Program to calculate factorial of a number using while loop

# Input from user
num = int(input("Enter a number: "))
fact = 1
i = 1

# While loop to calculate factorial
while i <= num:
    fact = fact * i
    i = i + 1

# Display result
print("Factorial of", num, "is:", fact)
```

Sample Output

```
Enter a number: 5
Factorial of 5 is: 120
```

Explanation: The while loop repeats until the condition $i \leq \text{num}$ becomes false, multiplying fact by i on every pass. The variable fact stores the running result.

Exercise 4: Nested if — Grading System

Note: Added to this manual: nested if statements are listed in the Week 3 syllabus but were not covered in the original draft.

Aim: Use a nested if statement to assign a grade based on marks scored in a test.

Program:

```
# Program to assign a grade using nested if statements

marks = int(input("Enter marks (out of 100): "))

if marks >= 35:
    if marks >= 90:
        grade = "A"
    else:
        if marks >= 75:
            grade = "B"
        else:
            grade = "C"
else:
    grade = "Fail"

print("Grade:", grade)
```

Sample Output

Enter marks (out of 100): 82
Grade: B

Explanation: The outer if first checks whether the student has passed (marks >= 35). Only if that is True does an inner (nested) if decide which grade band the marks fall into.

Exercise 5: Loop Control Statements — break, continue, pass

Note: Added to this manual: *break, continue, and pass* are listed in the Week 3 syllabus but were not covered in the original draft.

Aim: Demonstrate break, continue, and pass while scanning a list of machine sensor readings.

Program:

```
# Program to demonstrate break, continue, and pass

readings = [45, 50, -1, 60, 999, 55]

for value in readings:
    if value == -1:
        # faulty sensor reading -> skip it and continue
        continue
    elif value == 999:
        # critical fault -> stop scanning immediately
        print("Critical fault detected. Stopping scan.")
        break
    elif value > 100:
        pass # placeholder for future high-value handling
    else:
        print("Reading OK:", value)
```

Sample Output

```
Reading OK: 45
Reading OK: 50
Reading OK: 60
Critical fault detected. Stopping scan.
```

Explanation: continue skips the faulty (-1) reading; break stops the loop the moment a critical value (999) is found, so the reading after it (55) is never checked; pass is a placeholder that does nothing.

Additional Practice Programs

The following short programs were part of the original Week 3 draft. They are retained here, cleaned up and given sample outputs, as extra if-else and loop practice.

Part A: if-else — Practice Set

A.1 Positive / Negative / Zero

```
n = int(input())

if n > 0:
    print("Positive")
elif n < 0:
    print("Negative")
else:
    print("Zero")
```

Sample Output

```
5
Positive
```

Explanation: > checks positive, < checks negative, and 0 is handled separately by else.

A.2 Largest of Two Numbers

```
a = int(input())
b = int(input())

if a > b:
    print(a)
else:
    print(b)
```

Sample Output

```
15
9
15
```

Explanation: Compares the two values using > and prints the larger one.

A.3 Voting Eligibility

```
age = int(input())

if age >= 18:
    print("Eligible")
else:
    print("Not Eligible")
```

Sample Output

```
20
Eligible
```

Explanation: The minimum voting age is 18.

Part B: Loops — Practice Set

B.1 Print 1 to 10

```
for i in range(1, 11):  
    print(i)
```

Sample Output

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Explanation: range(1, 11) generates numbers 1 to 10, printed one per line.

B.2 Even Numbers from 1 to 50

```
for i in range(2, 51, 2):  
    print(i)
```

Sample Output

```
2  
4  
6  
8  
10  
...  
48  
50
```

Explanation: The third value in range(2, 51, 2) is the step size — stepping by 2 from 2 to 50 prints only even numbers.

B.3 Reverse a Number

```
n = int(input())  
rev = 0  
  
while n > 0:
```

```

    rev = rev * 10 + n % 10
    n //= 10

print(rev)

```

Sample Output

```

1234
4321

```

Explanation: $n \% 10$ extracts the last digit each time, which is appended to rev ; $n //= 10$ removes that digit from n .

B.4 Count Digits

```

n = int(input())
count = 0

while n > 0:
    count += 1
    n //= 10

print(count)

```

Sample Output

```

4321
4

```

Explanation: One digit is removed from n on every pass of the loop; $count$ keeps track of how many passes were needed.

Self-Practice (To Be Completed Individually)

Attempt the following on your own and get your output verified by the faculty:

- Write a Python program that uses if-elif-else to check whether a machine component's temperature is Normal, Warning, or Critical.
- Write a Python program using a for loop to print the sum of the first n natural numbers entered by the user.
- Write a Python program that uses a nested if to check whether a triangle (given three sides) is equilateral, isosceles, or scalene.
- Write a Python program that scans a list of numbers and uses continue to skip negative values while printing the rest.
- Write a Python program that uses break to stop a while loop as soon as a sensor reading exceeds a safe limit.

Reference: Week 3 syllabus content (25ME34I — Programming Concepts in Mechanical Engineering, DTE Karnataka).

PYTHON PROGRAMMING LAB MANUAL

Week 4 — Functions and Modules

Program	Mechanical Engineering	Semester	IV
Course Name	Programming Concepts in Mechanical Engineering	Course Code	25ME34I
Week	4	Duration	3 Hours (Practice Session)
Course Outcome	CO-1: Use appropriate syntaxes while writing program.	Program Outcomes	PO-1, PO-2, PO-4

Learning Objectives

- Define functions with parameters and return values, and call them to solve engineering problems.
- Understand the difference between local and global variable scope.
- Write short lambda (anonymous) functions for simple one-line tasks.
- Distinguish built-in functions from user-defined functions.
- Import and use standard Python modules such as math.

Quick Theory Recap

- Function definition: `def function_name(parameters): ...` — a reusable block of code that can accept inputs (parameters) and give back a result using `return`.
- Scope: a variable created inside a function is local to that function; a variable created outside any function is global and can be read anywhere.
- Lambda function: a short, unnamed function written as `lambda arguments: expression`, useful for small one-line operations.
- Built-in functions (e.g. `print()`, `len()`, `max()`) are ready-made and come with Python; user-defined functions are written by the programmer for a specific task.
- Modules: `import` brings in extra functionality, e.g. `import math` gives access to functions like `sqrt()`, `sin()`, `cos()`, and constants like `math.pi`.

Practical Exercises

Exercise 1: Area of a Circle Using a Function

Aim: Define a function that calculates and returns the area of a circle for a given radius.

Concept:

- Area of a circle = $\pi \times r^2$

Program:

```
# Program to calculate the area of a circle using a function

def area_of_circle(radius):
    pi = 3.14159
    area = pi * radius * radius
    return area

# Input from user
r = float(input("Enter the radius of the circle: "))

# Function call
result = area_of_circle(r)
print("Area of the circle is:", result)
```

Sample Output

```
Enter the radius of the circle: 7
Area of the circle is: 153.93791
```

Explanation: area_of_circle(radius) is a user-defined function with one parameter. It computes the area and sends the value back to the caller using return, where it is stored in result and printed.

Exercise 2: Greatest Common Divisor (GCD) Using a Function

Aim: Write a function that finds the GCD of two numbers entered by the user.

Concept:

- GCD (Greatest Common Divisor) is the largest number that divides both given numbers exactly.
- Euclid's method: repeatedly replace the larger number with the remainder of dividing it by the smaller number, until the remainder is 0.

Program:

```
# Program to find the GCD of two numbers using a function

def find_gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Input from user
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

# Function call
gcd = find_gcd(num1, num2)
print("GCD of", num1, "and", num2, "is:", gcd)
```

Sample Output

```
Enter first number: 36
Enter second number: 24
GCD of 36 and 24 is: 12
```

Explanation: `find_gcd(a, b)` repeatedly applies $a, b = b, a \% b$ (Euclid's algorithm) until b becomes 0; at that point a holds the GCD, which is returned to the caller.

Exercise 3: Lambda Functions — Squaring a Number

Aim: Explore lambda functions by writing a small one-liner that squares a number.

Program:

```
# Program to demonstrate a lambda function

# A lambda function to square a number
square = lambda x: x * x

# Input from user
n = float(input("Enter a number: "))

# Using the lambda function
print("Square of", n, "is:", square(n))
```

Sample Output

```
Enter a number: 6
Square of 6.0 is: 36.0
```

Explanation: `lambda x: x * x` defines a small, unnamed function in a single line and assigns it to `square`. It works exactly like a normal function defined with `def`, but is best suited for short, simple tasks.

Note: A few more one-liner lambda examples for practice: `cube = lambda x: x ** 3`, `is_even = lambda x: x % 2 == 0`, `add = lambda a, b: a + b`.

Exercise 4: Using the math Module for Trigonometric Calculations

Aim: Use Python's built-in math module to perform trigonometric calculations.

Program:

```
# Program to perform trigonometric calculations using the math module

import math

# Input angle in degrees
angle_deg = float(input("Enter angle in degrees: "))

# Convert degrees to radians (math functions expect radians)
angle_rad = math.radians(angle_deg)
```

```
# Trigonometric calculations
sin_val = math.sin(angle_rad)
cos_val = math.cos(angle_rad)
tan_val = math.tan(angle_rad)

# Display results
print("sin(", angle_deg, ") =", round(sin_val, 4))
print("cos(", angle_deg, ") =", round(cos_val, 4))
print("tan(", angle_deg, ") =", round(tan_val, 4))
```

Sample Output

```
Enter angle in degrees: 30
sin( 30.0 ) = 0.5
cos( 30.0 ) = 0.866
tan( 30.0 ) = 0.5774
```

Explanation: `import math` gives access to ready-made trigonometric functions. Since `math.sin()`, `math.cos()`, and `math.tan()` expect the angle in radians, `math.radians()` is used first to convert the user's degree input.

Self-Practice (To Be Completed Individually)

Attempt the following on your own and get your output verified by the faculty:

- Write a function `volume_of_cylinder(radius, height)` that returns the volume of a cylinder ($V = \pi \times r^2 \times h$).
- Write a function to find the Least Common Multiple (LCM) of two numbers (hint: $LCM = (a \times b) / GCD(a, b)$).
- Write a lambda function that converts a temperature from Celsius to Fahrenheit.
- Write a program using the `math` module to calculate the hypotenuse of a right triangle given the two other sides (`math.sqrt` and `math.pow`, or `math.hypot`).
- Write a function that takes a list of numbers and returns the largest value, without using the built-in `max()` function.

Reference: Week 4 syllabus content (25ME34I — Programming Concepts in Mechanical Engineering, DTE Karnataka).